

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Описание предметной области	4
1.2 Типы карт	5
1.3 Механика работы рынка карт	7
1.4 Категории пользователей	7
1.5 Требования к разрабатываемому приложению	10
1.6 Формализация данных	10
1.7 Выбор модели хранения данных	13
2 Конструкторский раздел	15
2.1 Схема базы данных	15
2.2 Ограничения целостности	15
2.3 Нормальная форма	16
2.3.1 Таблица пользователей	17
2.3.2 Таблица ролей	17
2.3.3 Таблица игр	18
2.3.4 Таблица игроков	18
2.3.5 Таблица карт	18
2.3.6 Таблица типов карт	19
2.3.7 Таблица колод	19
2.3.8 Таблица карт в колодах	20
2.3.9 Таблица изображений	20
2.3.10 Вывод	21
2.4 Проектирование хранимой процедуры покупки карты	21
2.5 Проектирование триггера	24
2.6 Ролевая модель	25
2.7 Вывод	26
3 Технологический раздел	27
4 Исследовательский раздел	28

ЗАКЛЮЧЕНИЕ	29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	30

ВВЕДЕНИЕ

1 Аналитическая часть

В аналитической части проведён анализ предметной области онлайн-версии настольной игры «Эпичные схватки боевых магов: Крутагидон», в том числе описание типов карт, механики работы рынка и условий победы. Сформулированы требования и ограничения к приложению, формализованы данные и роли пользователей.

1.1 Описание предметной области

Онлайн-версия упомянутой колодостроительной (deck-building) игры представляет собой многопользовательскую карточную игру, в которой игроки сражаются между собой, используя карты из общей системы карт. Для реализации используются механики, описанные в правилах игры, но некоторые объекты переименованы [1].

Игровой процесс строится на последовательности ходов *игроков* — участников игровой сессии, обладающих параметрами состояния (например, здоровье, очки крутости) и взаимодействующих с игровыми объектами в процессе игры. Во время своего хода игрок может разыгрывать карты из своей руки, получать игровой ресурс — *эхо* — и использовать его для покупки новых карт из рынка. Купленные карты добавляются в личную колоду игрока и могут быть использованы в дальнейшем. В конце каждого хода все карты, с которыми игрок начинал свой ход, сбрасываются, а на руку набираются новые [1].

Игрок обладает личной колодой карт, которая используется в процессе игры. Колода формируется из стартовых карт (началок), а затем пополняется новыми картами, приобретёнными игроком на рынке. Механика работы рынка приведена ниже. В ходе игры карты могут находиться в различных состояниях (колодах): рука, добор, стол, сброс, рынок, основная колода и игровой сброс [1].

Типы колод:

- рука (hand) — просматриваемые и разыгрываемые карты, с которыми игрок начинает ход;
- колода добора (draw) — колода игрока, из которой формируется рука и осуществляется добор карт;

- стол (table) — карты, которые игрок разыграл на текущем ходу, доступны к просмотру всем игрокам;
- сброс (discard) — колода для сброшенных карт, формирует колоду добора, когда в ней заканчиваются карты;
- рынок (market) — колода из 5 открытых карт, которые доступны для просмотра всем игроками и покупки игроку в свой ход;
- основная игровая колода (deck) — колода, из которой формируется рынок;
- сброс игры (banish) — карты, которые сбрасываются без возврата в игру.

В течение ходов игроки могут получать урон от атак других игроков или атаковать самостоятельно, пополнять руку новыми картами, лечиться, набирать внутриигровую валюту покупки карт — эхо. Атаки уменьшают количество очков здоровья игрока и, если здоровье игрока становится меньше или равно 0, то игрок на этом ходу умирает, получает «памятку о бренности бытия», отмечающую его смерть, и сразу же восстанавливается со стандартным количеством очков здоровья [1].

Игра заканчивается, когда все выделенные на игру памятки о бренности бытия заканчиваются. Победитель определяется подсчётом очков крутости, которые складываются из очков крутости каждой карты, которой обладает игрок на момент завершения игры [1].

Для корректной работы системы необходимо хранить информацию об игроках, текущих партиях, составе колод игроков и колод игры, картах и типах карт.

1.2 Типы карт

Карты являются основным элементом для реализации игровой механики. Каждая карта обладает определённым набором характеристик и эффектов и принадлежит определённому типу.

В системе используются следующие типы карт: началка, существа (дварф, эльф, дракон, волшебник), штуки и чертоги.

Началка — карты стартовой колоды игроков.

Дварф, эльф, дракон, и волшебник — карты существ с определёнными характеристиками.

Штука — карты предметов с определёнными характеристиками.

Чертог — карты локаций, дающие постоянные эффекты.

Карты могут обладать следующими эффектами: защита, добор, атака, хилл, буст руки и буст эхо.

Защита — позволяет избежать атаки другого игрока.

Добор — позволяет взять и разыграть дополнительные карты.

Атака — позволяет атаковать другого игрока.

Хилка — позволяет восстановить часть здоровья.

Буст руки — позволяет разыгрывать больше карт, действует постоянно.

Буст эхо — увеличивает базовое значение ресурса эхо игрока, действует постоянно.

Каждая карта имеет следующие свойства:

- название;
- тип карты;
- изображение;
- эффект карты и его описание;
- числовая характеристика эффекта;
- значение эхо;
- стоимость для покупки;
- количество очков крутости.

1.3 Механика работы рынка карт

В каждой партии формируется рынок карт, из которого игроки могут приобретать новые карты. Рынок состоит из пяти открытых карт, доступных для покупки, и закрытой колоды, из которой рынок обновляется в начале каждого хода [1].

После розыгрыша своих карт игрок может купить любое количество открытых карт рынка, на которое у него хватает накопленного эхо. После покупки карта удаляется из рынка и добавляется в личную колоду игрока [1].

На покупку карт накладываются следующие ограничения:

- нельзя купить карт больше, чем есть открытых карт на рынке;
- одну и ту же карту нельзя купить больше одного раза;
- нельзя купить карту, отсутствующую на открытом рынке.

Для покупки карт используется накапливаемое игроком в течение хода эхо. Каждая разыгранная карта может как увеличить, так и уменьшить или не изменить текущее значение эхо [1].

Значение эхо хранится для каждого игрока отдельно, накапливается в течение хода и после него сбрасывается до нуля (или базового значения конкретного игрока, если он использовал карты, меняющие этот показатель) [1].

При покупке карт с рынка текущее значение эхо уменьшается на стоимость купленной карты. Карта может быть куплена только в том случае, когда текущее значение эхо игрока не меньше стоимости карты [1].

1.4 Категории пользователей

В системе предусмотрено несколько категорий пользователей, обладающих разными правами доступа:

Неавторизованный пользователь (гость) — пользователь, который может просматривать правила игры и каталог карт.

Авторизованный пользователь — пользователь, который может делать то же самое, что и неавторизованный, а также может создавать игры или присоединяться к уже созданным.

Игрок — пользователь, находящийся в игровой партии, способный выполнять ходы.

Модератор — пользователь, обладающий всеми возможностями авторизованного пользователя, который может блокировать и разблокировать других пользователей, просматривать текущие игры, временно их останавливать или завершать.

Мастер — пользователь, который может делать всё то же, что и модератор, а также добавлять новые карты, менять или удалять уже существующие.

Диаграмма вариантов использования разрабатываемой системы представлена на рисунке 1.1.

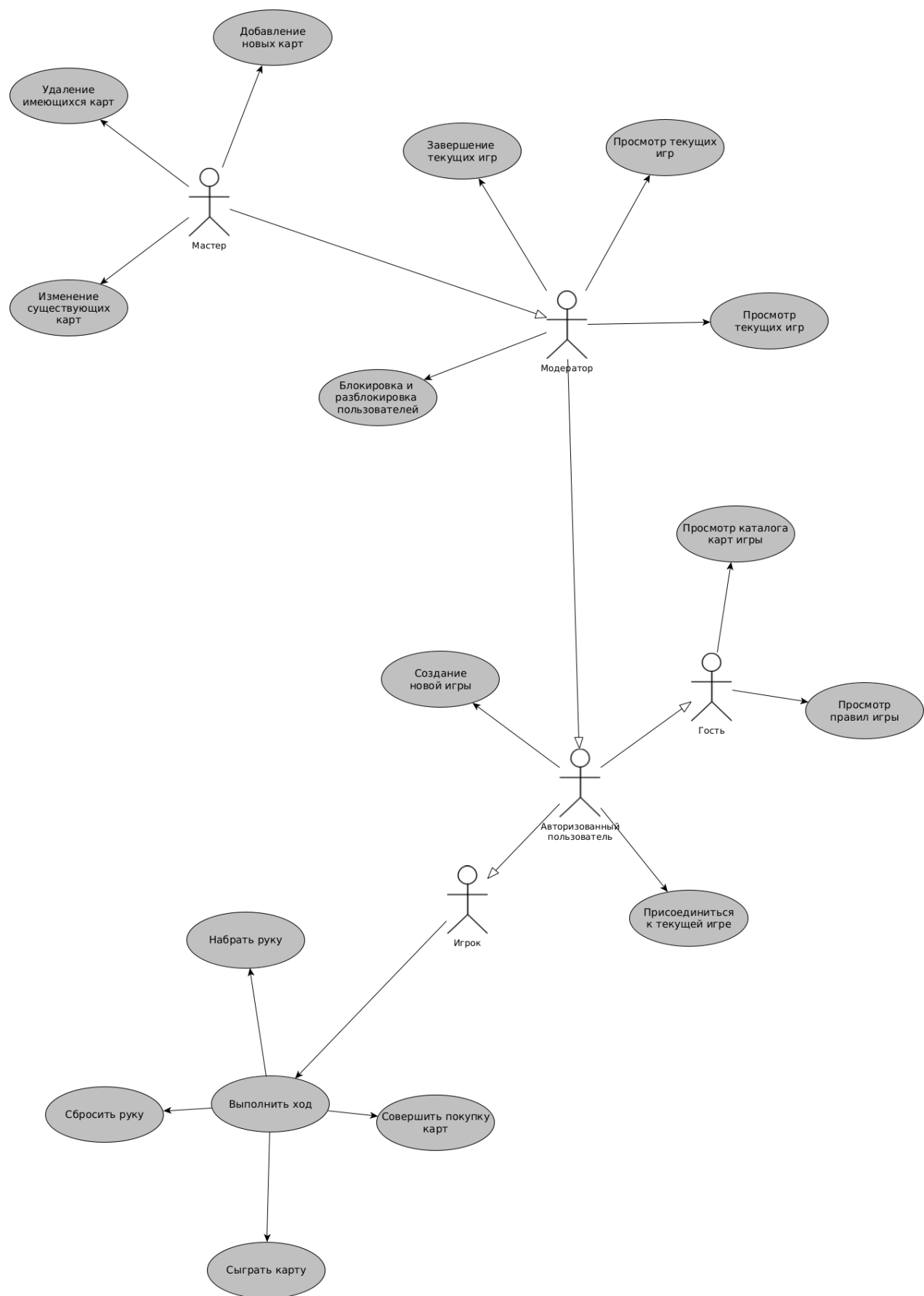


Рисунок 1.1 – Диаграмма вариантов использования разрабатываемой системы

1.5 Требования к разрабатываемому приложению

Разрабатываемый сайт предназначен для одного или нескольких гостей, способных просматривать игровые карты и правила игры, и пользователей, способных авторизоваться и быть участниками игровых партий.

Таким образом, разрабатываемое приложение должно обладать следующей функциональностью:

1. управление игровыми сессиями (создание, подключение и завершение игры);
2. взаимодействие игроков в рамках игровой сессии;
3. отображение текущего состояния игры (поле, карты, колоды и прочие игровые сущности);
4. выполнение игровых действий пользователем (применение эффектов карт, покупка новых);
5. обработка логики игры (подсчёт очков, применение правил, переходы между состояниями);
6. поддержка работы с колодами карт (формирование, перемешивание, добор и сброс карт);

Таким образом, приложение должно обеспечивать полный цикл управления игровыми сессиями, включая создание, участие в игре и просмотр результатов.

1.6 Формализация данных

В таблице 1.1 представлены сущности, используемые в базе данных.

Таблица 1.1 – Сущности и их свойства

Сущность	Свойство	Метка	Тип	Атрибут БД
Пользователь	Псевдоним	Имя пользователя, уникально	Строка	username

Таблица 1.1 – Сущности и их свойства

Сущность	Свойство	Метка (Констрейнты)	Тип	Атрибут БД
	Эл. почта	Адрес почты, уникально	Строка	e-mail
	Пароль	Хеш пароля	Строка	password
	Дата рег.	Время создания аккаунта	DateTime	register_date
Игра	Текущий ход	Номер текущего хода	Целое число	current_move
	Памятки	Оставшееся кол-во памяток	Целое число	memos
	Время начала	Момент старта партии	DateTime	time_start
	Победитель	Внешний ключ к Players, может быть NULL	Целое число	winner
Игрок	Пользователь	Внешний ключ к Users	Целое число	user_id
	Игра	Внешний ключ к Games	Целое число	game_id
	Здоровье	Текущее здоровье	Целое число	health
	Лимит руки	Макс. карт в руке	Целое число	hand_limit
	Базовое эхо	Базовый ресурс	Целое число	base_echo
	Порядок хода	Очередность хода игрока	Целое число	turn
Карта	Название	Название карты	Строка	title
	Эхо	Количество эхо карты	Целое число	echo
	Стоимость	Цена покупки	Целое число	cost

Таблица 1.1 – Сущности и их свойства

Сущность	Свойство	Метка (Констрейнты)	Тип	Атрибут БД
	Сила	Значение эффекта	Целое число	power
	Тип карты	Внешний ключ к Types	Целое число	card_type
	Очки крутости	Очки за карту	Целое число	cool_points
Тип карты	Действие	Тип эффекта	Строка	action
	Постоянность	Признак постоянного эффекта	Логический	if_permanent
Колода	Тип	Тип (market, hand и т.д.)	Строка	type
	Открытость	Могут карты быть просмотрены или нет	Логический	if_open
	Владелец	Внешний ключ к Players, может быть NULL	Целое число	owner_id
Карта в колоде	Колода	Внешний ключ к Decks	Целое число	deck_id
	Карта	Внешний ключ к Cards	Целое число	card_id
	Позиция	Порядковый номер карты в стопке	Целое число	position
Изображение	Имя	Имя файла/заголовка	Строка	title
	Файл	Путь к файлу изображения	Path	file

На рисунке 1.2 представлена диаграмма сущность-связь, отражающая

структуру базы данных и связи между сущностями.

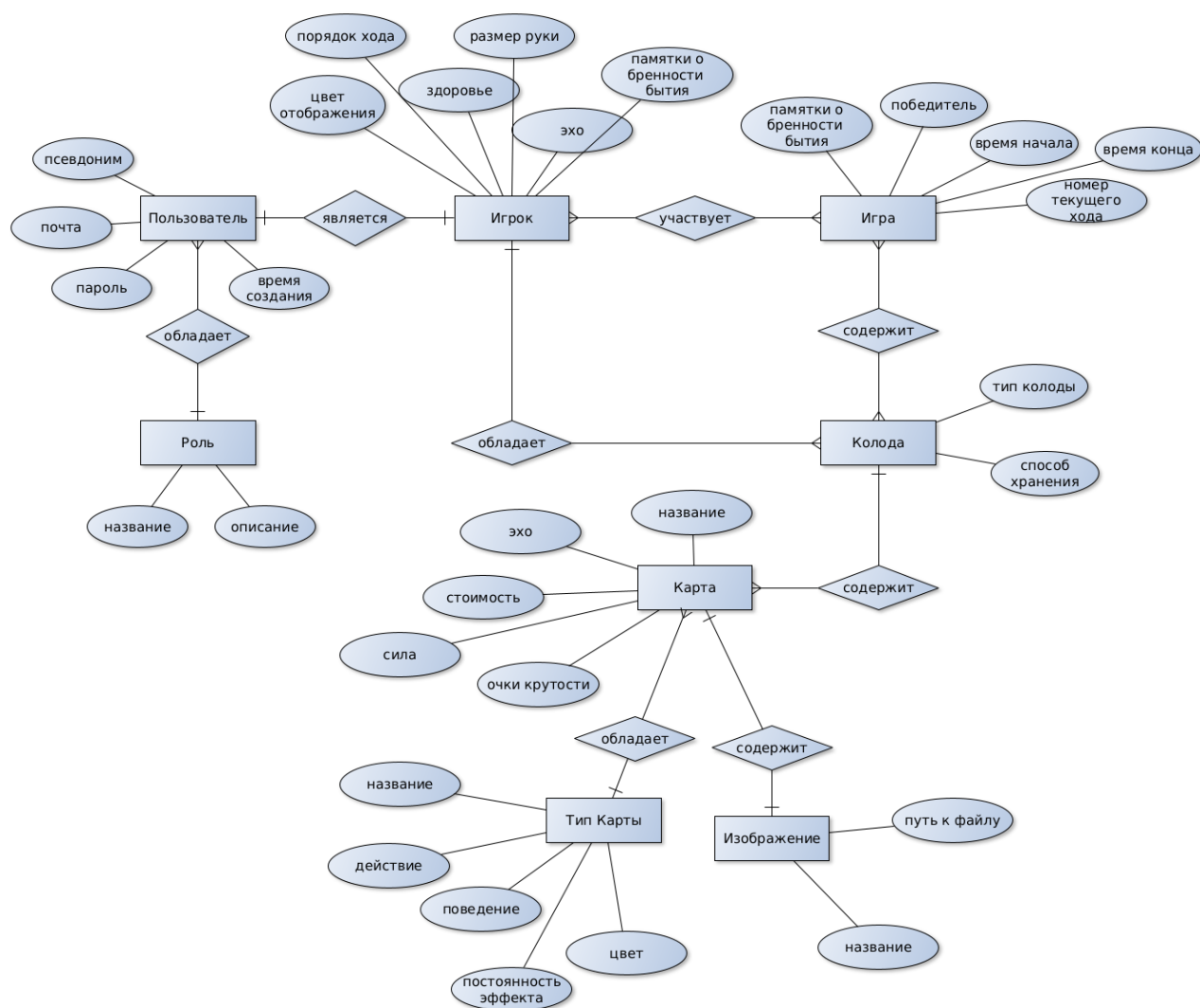


Рисунок 1.2 – Диаграмма сущность-связь в нотации Чена

1.7 Выбор модели хранения данных

В разрабатываемой системе предполагается хранение структурированных данных предметной области, включающих сущности (например, пользователи, игровые объекты, карты) и связи между ними. Для каждой сущности задан фиксированный набор атрибутов и предусмотрены взаимосвязи, требующие поддержания целостности и согласованности данных [2].

Существует несколько основных моделей хранения данных:

Дореляционная модель — данные хранятся в виде иерархических структур, файлов или сетевых баз данных. Эта модель не предполагает жёсткой схемы таблиц и характеризуется сложностью в обеспечении ссылочной целостности. Операции с данными требуют специализированного

программного обеспечения, а структура данных тесно связана с применением. Дореляционный подход хорошо подходит для специализированных приложений, но неудобен для систем, требующих гибкости и универсальности [2].

Реляционная модель — информация представляется в виде таблиц с явно определёнными атрибутами, первичными и внешними ключами, ограничениями целостности и уникальности. Реляционная модель обеспечивает строгое разделение структуры данных от логики приложения, позволяет использовать стандартный язык запросов (SQL), удобна для сложных операций и обеспечивает надёжность данных через механизмы транзакций [3].

Постреляционная модель — Такие БД основаны на реляционных, однако отходят от их принципов в пользу более эффективного решения специфических задач. Одним из примеров таких особенностей выступает возможность вложенного хранения таблиц (хранения таблиц в таблицах), что невозможно в реляционных БД [4].

С учётом требований разрабатываемой системы наиболее подходящей является реляционная модель данных. Она позволяет явно определить структуру данных (пользователи, игры, игроки, карты, колоды), обеспечить строгое соблюдение ограничений целостности, поддерживать сложные многотабличные запросы для анализа состояния игры и гарантировать консистентность данных при одновременном доступе нескольких игроков. Реляционная модель удобна для выполнения сложных запросов и фильтрации информации.

Вывод

В аналитической части была рассмотрена предметная область, были рассмотрены и формализованы данные, подлежащие хранению и выбрана реляционная модель хранения данных.

- стоимость карты, значение эхо, количество очков крутости и сила эффекта карты не могут быть отрицательными;
- тип колоды должен принимать одно из допустимых значений: колода добора, рука, стол, сброс игрока, общий игровой сброс, открытый рынок или закрытый рынок;
- в рамках одной партии у одного игрока не должно быть более одной колоды каждого личного типа: колоды добора, руки, стола и сброса;
- в рамках одной партии не должно быть более одной открытой колоды рынка, одной закрытой колоды рынка и одной колоды игрового сброса;
- номера позиций карт должны быть уникальными в рамках одной колоды и иметь неотрицательные значения;
- купить можно только карту, находящуюся в открытом рынке текущей партии;
- покупка карты возможна только при достаточном количестве эхо у игрока;
- после покупки карта должна быть удалена из открытого рынка и добавлена в сброс игрока;
- если количество оставшихся памяток о бренности бытия становится равным нулю, партия должна перейти в завершённое состояние.

2.3 Нормальная форма

Для проверки корректности проектирования базы данных была выполнена проверка отношений на соответствие третьей нормальной форме (3НФ) [3].

Отношение находится в третьей нормальной форме, если:

- оно находится во второй нормальной форме;
- каждый неключевой атрибут нетранзитивно зависит от первичного ключа.

2.3.1 Таблица пользователей

Отношение:

Users(id, username, email, password, register_date, role)

Функциональные зависимости:

$id \rightarrow username, email, password, register_date, role$

Также:

$username \rightarrow id$

$email \rightarrow id$

Первичный ключ:

id

Кандидатные ключи:

username, email

Все неключевые атрибуты полностью функционально зависят от первичного ключа. Транзитивные зависимости отсутствуют.

Следовательно, отношение **Users** находится в 3НФ.

2.3.2 Таблица ролей

Отношение:

Roles(id, role)

Функциональные зависимости:

$id \rightarrow role$

Первичный ключ:

id

Транзитивные зависимости отсутствуют.

Следовательно, отношение **Roles** находится в 3НФ.

2.3.3 Таблица игр

Отношение:

Games(id, current_move, memos, time_start, time_end, winner)

Функциональные зависимости:

$id \rightarrow current_move, memos, time_start, time_end, winner$

Первичный ключ:

id

Все атрибуты зависят только от первичного ключа. Транзитивные зависимости отсутствуют.

Следовательно, отношение **Games** находится в 3НФ.

2.3.4 Таблица игроков

Отношение:

Players(id, game_id, user_id, color, health, hand_limit, base_echo, memos, turn)

Функциональные зависимости:

$id \rightarrow game_id, user_id, color, health, hand_limit, base_echo, memos, turn$

Первичный ключ:

id

Все неключевые атрибуты полностью зависят от первичного ключа. Транзитивные зависимости отсутствуют.

Следовательно, отношение **Players** находится в 3НФ.

2.3.5 Таблица карт

Отношение:

Cards(id, title, creature, echo, cost, power, card_type, image_id, cool_points)

Функциональные зависимости:

$$id \rightarrow title, creature, echo, cost, power, card_type, image_id, cool_points$$

Первичный ключ:

$$id$$

Все атрибуты полностью функционально зависят от первичного ключа. Транзитивные зависимости отсутствуют.

Следовательно, отношение **Cards** находится в 3НФ.

2.3.6 Таблица типов карт

Отношение:

$$Types(id, action, usage_pattern, if_permanent, color)$$

Функциональные зависимости:

$$id \rightarrow action, usage_pattern, if_permanent, color$$

Первичный ключ:

$$id$$

Все атрибуты зависят только от первичного ключа. Транзитивные зависимости отсутствуют.

Следовательно, отношение **Types** находится в 3НФ.

2.3.7 Таблица колод

Отношение:

$$Decks(id, type, if_open, owner_id)$$

Функциональные зависимости:

$$id \rightarrow type, if_open, owner_id$$

Первичный ключ:

id

Все атрибуты зависят только от первичного ключа. Транзитивные зависимости отсутствуют.

Следовательно, отношение **Decks** находится в 3НФ.

2.3.8 Таблица карт в колодах

Отношение:

DecksCards(id, deck_id, card_id, position)

Функциональные зависимости:

$id \rightarrow deck_id, card_id, position$

Дополнительно:

$(deck_id, position) \rightarrow card_id$

Первичный ключ:

id

Альтернативный ключ:

$(deck_id, position)$

Все атрибуты полностью функционально зависят от ключей. Транзитивные зависимости отсутствуют.

Следовательно, отношение **DecksCards** находится в 3НФ.

2.3.9 Таблица изображений

Отношение:

Images(id, title, file)

Функциональные зависимости:

$$id \rightarrow title, file$$

Первичный ключ:

$$id$$

Все атрибуты полностью зависят от первичного ключа. Транзитивные зависимости отсутствуют.

Следовательно, отношение **Images** находится в 3НФ.

2.3.10 Вывод

Во всех отношениях:

- отсутствуют повторяющиеся группы атрибутов;
- все неключевые атрибуты полностью функционально зависят от ключей;
- отсутствуют транзитивные зависимости между неключевыми атрибутами.

Следовательно, отношения базы данных находятся в третьей нормальной форме.

2.4 Проектирование хранимой процедуры покупки карты

Для выполнения операции покупки карты с открытого рынка в базе реализована хранимая процедура *buy_card_from_market*. Она используется для того, чтобы все проверки и все изменения происходили вместе.

Атомарность операции покупки карты обеспечивается механизмом транзакций СУБД. Все изменения данных выполняются в рамках одной транзакции. В случае возникновения ошибки на любом этапе выполнения процедуры формируется исключение, что приводит к откату транзакции. Таким образом, либо все изменения (списание эхо, удаление карты из рынка и добавление карты в сброс) выполняются полностью, либо не выполняется ни одно из них.

Транзакционность операции обеспечивается тем, что выполнение хранимой процедуры происходит в рамках одной транзакции СУБД. Все проверки

и изменения состояния выполняются последовательно и фиксируются только при успешном завершении процедуры. В случае возникновения ошибки транзакция откатывается.

Схема хранимой процедуры представлена на рисунке 2.2.

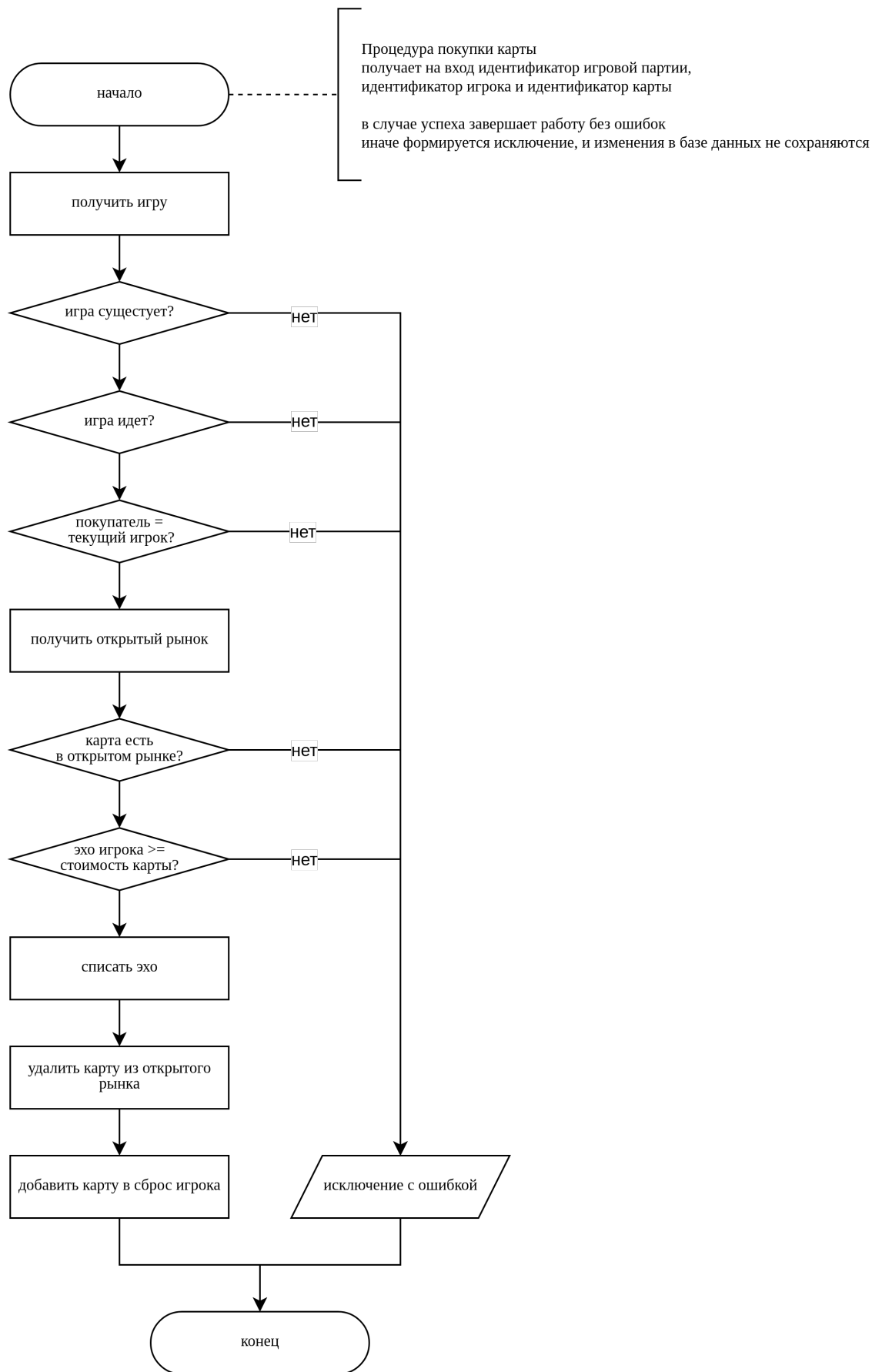


Рисунок 2.2 – Схема хранимой процедуры покупки карты из открытого рынка

2.5 Проектирование триггера

Для поддержания целостности данных необходимо, чтобы тип колоды соответствовал её владельцу. Личные колоды игрока, такие как колода добора, рука, стол и сброс, должны принадлежать конкретному игроку. Общие колоды партии, такие как открытый рынок, закрытый рынок и общий игровой сброс, не должны иметь владельца — они принадлежат игре напрямую.

Для проверки этого ограничения используется триггер, который срабатывает перед добавлением или изменением записи в таблице **decks**. Триггер проверяет тип колоды, наличие владельца и соответствие партии колоды партии игрока.

Схема работы такого триггера представлена на рисунке 2.3.

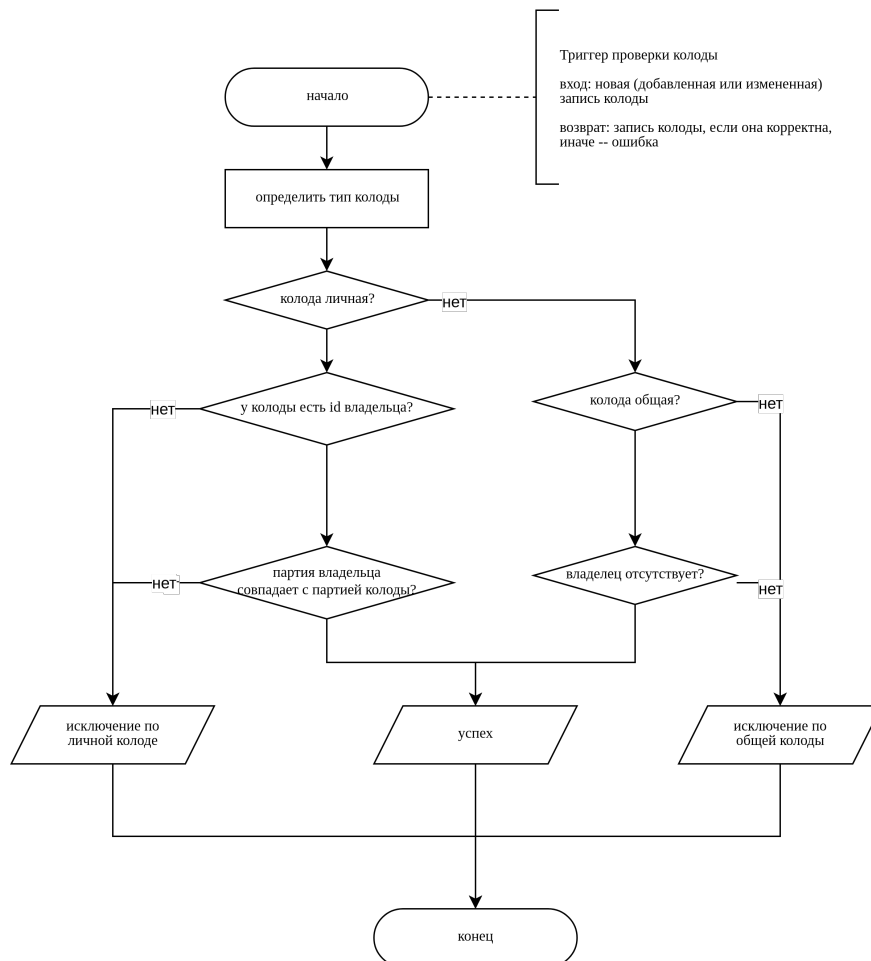


Рисунок 2.3 – Схема триггера проверки владельца колоды

2.6 Ролевая модель

На основе пользователей приложения, описанных в аналитической части, были сформированы следующие роли и права доступа к базе данных:

- пользователь (гость) — имеет доступ к каталогу игровых карт и правилам;
- авторизованный пользователь — так же может просматривать и создавать записи в таблице игровых партий;
- игрок — имеет доступ к просмотру, изменению записей таблиц игровых колод и просмотру таблицы игроков в игре;
- модератор — обладает всеми правами авторизованного пользователя и может изменять и удалять записи в таблицах пользователей и игровых партий;
- мастер — всё то же, что и модератор, а также изменение и удаление записей в каталоге карт.

Описание прав доступа к таблицам на создание, чтение, изменение и удаление данных (CRUD) представлено в таблице 2.1.

В таблице используются следующие сокращения:

- польз. — пользователь;
- автор. польз. — авторизированный пользователь.

Таблица 2.1 – Права доступа к таблицам

Таблица	Польз.	Автор. польз.	Игрок	Модератор	Мастер
Пользователь	-	-	-	CRUD	CRUD
Игра	-	CR	R	CRUD	CRUD
Игрок	-	CR ¹	R	CRUD	CRUD
Карта	R	R	R	R	CRUD

¹ Только для создания записи о собственном участии в партии

Тип карты	R	R	R	R	CRUD
Изображение	R	R	R	R	CRUD
Колода	-	-	R	R	R
Карта в колоде	-	-	RUD ²	R	R

2.7 Вывод

В результате конструкторского раздела была разработана реляционная схема базы данных, рассмотрены ограничения целостности на неё, описана ролевая модель, реализованы хранимая процедура покупки карты и триггер проверки колоды.

² Изменение выполняется только в рамках игровых операций или хранимых процедур

3 Технологический раздел

4 Исследовательский раздел

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Мосигра*. Правила игры «Эпичные схватки боевых магов. Крутагидон». — URL: <https://www.mosigra.ru/download/rules/jepichnie-shvatki-boevih-magov-krutagidon-2025-rules.pdf> (дата обращения: 20.02.2026).
2. Конспекты лекций курса «Базы данных» за 2024 год. МГТУ им. Н.Э. Баумана, кафедра «Программное обеспечение ЭВМ и информационные технологии». — Учебные материалы.
3. Дейт, К. Дж. Введение в системы баз данных. — 8-е изд. — М.: Издательский дом “Вильямс”, 2005. — 1328 с.
4. Парфенов, Ю. П. Постреляционные хранилища данных : учеб. пособие. — Екатеринбург: изд-во Урал, 2016. — 120 с.